

个人作业报告

题目：**并行程序设计与算法**

理论课第五次作业

姓名	林宏宇
学号	23320093
院系	计算机学院
专业	计算机科学与技术
指导教师	吴迪

2026 年 6 月 15 日

【摘　要】

此作业报告是并行程序设计与算法课程的**第五次作业**，作业题目来自智算习堂[1]的作业内容。

关键词：　并行程序设计与算法

目录

1 第一题 1
1.1 题目 1
1.2 答案 1

2 第二题 3
2.1 题目 3
2.2 答案 3

3 第三题 5
3.1 题目 5
3.2 答案 5

4 第四题 8
4.1 题目 8
4.2 答案 8

参考文献 10

1 第一题

1.1 题目

CUDA 程序中常见的内存类型包括全局内存、共享内存、寄存器、本地内存、常量内存等。请结合下面代码，回答以下问题：

```
__constant__ float c_alpha;
__device__ float d_total;

__global__ void memoryDemo(float* g_in, float* g_out, int n){
    __shared__ float s_tile[256];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    float temp = 0.0f;
    float localArray[4];

    if(idx < n){
        s_tile[tid] = g_in[idx];
        __syncthreads();
        temp = s_tile[tid] * c_alpha;
        localArray[0] = temp;
        g_out[idx] = localArray[0];
        atomicAdd(&d_total, temp);
    }
}
```

回答下列问题：

1) s_tile 属于哪类内存类型，简要描述该类内存的访问范围与生命周期
2) c_alpha 属于哪类内存？适合存放什么数据？

1.2 答案

1.2.1 s_tile 的内存类型及特性

s_tile 属于**共享内存**（Shared Memory）。

访问范围与生命周期：

- **访问范围**：同一个线程块（block）内的所有线程都可以访问
- **生命周期**：从线程块启动时分配，到线程块执行结束时释放
- **访问速度**：比全局内存快得多，延迟低，带宽高
- **容量限制**：每个 SM 的共享内存有限（通常几十 KB）
- **同步要求**：使用 __syncthreads() 确保所有线程完成写入后再读取

1.2.2 c_alpha 的内存类型及适用场景

c_alpha 属于**常量内存**（Constant Memory）。

适合存放的数据：

- **只读数据**：在核函数执行期间不会被修改
- **广播数据**：需要被多个线程同时访问的相同数据
- **小规模数据**：常量内存容量有限（通常 64KB）
- **频繁访问的数据**：常量内存有专门的缓存，访问速度快
- **示例**：标量系数、配置参数、查找表等

2 第二题

2.1 题目

阅读下面 CUDA 程序片段，回答以下问题：

```
__global__ void vecAdd(float* a, float* b, float* c, int n){
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    c[i] = a[i] + b[i];
}

int main(){
    int n = 1000;
    size_t size = n * sizeof(float);

    float* d_a, *d_b, *d_c;
    cudaMalloc((void**)&d_a, size);
    cudaMalloc((void**)&d_b, size);
    cudaMalloc((void**)&d_c, size);

    int threads = 256;
    int blocks = (n + threads - 1) / threads;

    vecAdd<<<blocks, threads>>>(d_a, d_b, d_c, n);
}
```

1) 这段核函数存在什么潜在的错误?
2) 当 n=1000 时，一共启动多少个线程？是否可能发生对数组的越界访问?

2.2 答案

2.2.1 核函数的潜在错误

核函数存在的潜在错误是**缺少边界检查**：

- 当线程总数不是 n 的整数倍时，部分线程的索引 i 会超过数组边界
- 例如：n=1000，threads=256，blocks=4，总线程数 =1024

- 索引范围为 0-1023，但有效数据只有 0-999
- 索引 1000-1023 的线程会访问越界内存，导致未定义行为

正确的核函数应该包含边界检查：

```
__global__ void vecAdd(float* a, float* b, float* c, int n){
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if(i < n){  // 添加边界检查
        c[i] = a[i] + b[i];
    }
}
```

2.2.2 线程数量及越界访问分析

当 n=1000 时：

- **线程总数**：blocks = (1000 + 256 - 1) / 256 = 4，总线程数 = 4 × 256 = 1024
- **越界访问**：是，会发生越界访问
- 索引范围：0-1023
- 有效数据范围：0-999
- 越界线程数：1024 - 1000 = 24 个线程（索引 1000-1023）
- 这些线程会访问 a[1000]-a[1023]、b[1000]-b[1023]、c[1000]-c[1023]，超出分配的内存范围

3 第三题

3.1 题目

假设有一个 N × N 的矩阵 M 存储在一维数组 float* mat 中（按行优先存储），要在 GPU 上做矩阵转置，kernel 如下：

```
__global__ void Transpose(float* mat, float* matT, int N){
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if(row < N && col < N){
        matT[col * N + row] = mat[row * N + col];
    }
}
```

主机端启动方式：

```
dim3 blockSize(16, 16);
dim3 gridSize((N + blockSize.x - 1) / blockSize.x,
              (N + blockSize.y - 1) / blockSize.y);
Transpose<<<gridSize, blockSize>>>(mat, matT, N);
```

请回答：

1) 如果 N=1000，计算 gridSize.x 和 gridSize.y 各是多少？
2) 假设 GPU 有 1024 个线程每个 SM 最多可以同时执行，blockSize 设为 (16,16) 后，每个 block 占用多少个线程？最后一个 block 中可能有多少线程不执行计算？
3) 如果去掉 kernel 中的边界判断 if(row<N && col<N)，会发生什么问题？为什么这个判断很重要？

3.2 答案

3.2.1 gridSize 的计算

当 N=1000，blockSize=(16,16) 时：

- gridSize.x = (1000 + 16 - 1) / 16 = 1015 / 16 = **63**

- gridSize.y = (1000 + 16 - 1) / 16 = 1015 / 16 = **63**
- 总 block 数 = 63 × 63 = 3969 个 block

3.2.2 线程占用及无效线程数

- **每个 block 的线程数**：16 × 16 = **256 个线程**
- **每个 SM 可同时执行的 block 数**：1024 / 256 = 4 个 block
- **最后一个 block 的无效线程数**：
  - 最后一个 block 对应的坐标范围为：

$$row = 992 \sim 1007, \quad col = 992 \sim 1007$$

  - 由于矩阵大小为 1000×1000，因此有效范围为：

$$row = 992 \sim 999, \quad col = 992 \sim 999$$

  - 有效线程数：

$$8 \times 8 = 64$$

  - block 总线程数：

$$16 \times 16 = 256$$

  - 因此无效线程数：

$$256 - 64 = 192$$

  - 最后一个 block 中有 **192 个线程**不会执行计算。

3.2.3 边界判断的重要性

去掉边界判断会导致的问题：

- **越界访问**：当 row≥N 或 col≥N 时，会访问 mat 和 matT 数组的非法内存
- **数据损坏**：可能覆盖其他内存区域的数据
- **程序崩溃**：访问未分配的内存会导致段错误
- **结果错误**：转置结果包含无效数据

边界判断的重要性：

- 确保只处理有效数据范围内的元素
- 防止内存越界访问

- 保证程序的正确性和稳定性
- 允许使用非整除的 grid 尺寸，提高灵活性

4 第四题

4.1 题目

假设有一个 N×N 的矩阵 M 存储在一维数组 float* mat 中，按行优先方式存储。现在希望在 GPU 上实现” 把矩阵中每个元素都乘以 2” 的操作，kernel 框架如下：

```
__global__ void MatScale2(float* mat, int N){
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if(________________________){
        mat[________________________] = 2.0f * mat[________________________];
    }
}
```

主机端启动方式如下：

```
dim3 blockSize(16, 16);
dim3 gridSize((N + blockSize.x - 1) / blockSize.x,
              (N + blockSize.y - 1) / blockSize.y);
MatScale2<<<gridSize, blockSize>>>(mat, N);
```

请完成下列问题：

1) 如果某线程满足 row=3，col=5，那么它处理的是矩阵中的哪一个元素？
2) 补全上面的 kernel，使每个线程对自己负责的矩阵元素乘以 2
3) 若 N=1000，计算 gridSize.x 和 gridSize.y 的值，并说明为什么 kernel 中必须写边界判断

4.2 答案

4.2.1 线程处理的元素位置

当 row=3，col=5 时：

- 这是矩阵中第 4 行（row 从 0 开始）、第 6 列（col 从 0 开始）的元素
- 在一维数组中的索引位置：row × N + col = 3 × N + 5
- 例如，如果 N=1000，则索引位置为：3 × 1000 + 5 = **3005**

4.2.2 补全 kernel 代码

```
__global__ void MatScale2(float* mat, int N){
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if(row < N && col < N){  // 边界检查
        int idx = row * N + col;  // 计算一维索引
        mat[idx] = 2.0f * mat[idx];  // 原地乘以2
    }
}
```

4.2.3 gridSize 计算及边界判断必要性

当 N=1000，blockSize=(16,16) 时：

- gridSize.x = (1000 + 16 - 1) / 16 = 1015 / 16 = **63**
- gridSize.y = (1000 + 16 - 1) / 16 = 1015 / 16 = **63**

kernel 中必须写边界判断的原因：

- **防止越界访问**：总线程数 = 63 × 63 × 256 = 1,016,064，但有效元素只有 1,000,000 个
- **避免无效计算**：超出边界的线程不应该修改内存
- **保证正确性**：确保只处理矩阵内的有效数据
- **防止非法内存访问**：访问超出数组范围的内存会导致结果错误，严重时可能引发程序运行失败
- **提高效率**：让无效线程快速退出，减少资源浪费

参考文献

[1] [Z]. https://easyai.sysu.edu.cn.